Oracle — CTF Writeup

Oracle — CTF Writeup

Platform: Perfect Root (AfricaHackon) **Room:** Oracle **Author:** Dr Programmer **Difficulty:** Medium **Points:** 150 **Category:** Cryptography — CBC Padding Oracle **Result:** Solved (1/1 flag captured)

Description

> You have done so well to get here, now the Oracle will answer any question you might have, goodluck.

Three Linux binaries, two ciphertexts. The setup is a classic CBC padding oracle: a black-box decryptor that tells you — silently, via exit code — whether the decrypted plaintext has valid PKCS#7 padding. By observing that one bit, you recover the full plaintext byte-by-byte without ever knowing the AES key.

Flag

`r00t{0r4cl3_p4dd1ng_6512e251d6e28884bd5b37a0}`

Files Provided

```
check          (ELF, stripped)  - the final scoring binary
validator      (ELF, stripped)  - formula validator (a decoy path)
oracle         (ELF, stripped)  - the padding oracle
ciphertext.bin  (64 bytes)      - IV(16) || C1(16) || C2(16) || C3(16)
ciphertext2.bin (80 bytes)      - IV(16) || C1..C4 - decoy/random
```

Static Recon

`strings` revealed:

- `oracle` contains a **fake flag**: `r00t{ORACLE_FAKE_TH3_P4DD1NG_IS_NOT_H3RE}`. A direct hint from the author: the answer is not in the binary — you must run the oracle attack.
- `oracle` usage: `Usage: %s [--mode 2] ciphertext.bin`
- `validator` prints `VALID`, `Formula verified: %s`, and `INVALID - formula mismatch` — it wants a derived formula, not raw plaintext.
- `check` prints `CORRECT` then `Flag: %s` — the scoring path.

The Oracle's Interface

Run with no args it prints usage. Run with a ciphertext:

```
$ ./oracle ciphertext.bin ; echo $?
0
$ ./oracle --mode 2 ciphertext.bin ; echo $?
1
```

No output on stdout. The result is communicated **only** through the process exit code. `0` = padding valid, non-zero = padding invalid. This is the leak.

`--mode 2` produced exit code 1 for the same input. In practice mode 2 is the "does not treat the leading 16 bytes as IV" variant; for the attack we needed mode 1.

Locating the Padding Byte

PKCS#7 padding means the last byte of the decrypted plaintext equals the number of padding bytes (1..16). Flipping bytes in the ciphertext will destroy the padding unless we flip bytes that are *not* part of the padding we're trying to preserve.

We flipped each byte of `ciphertext.bin` by `0x01` and checked the exit code:

```
positions where flipping breaks validity:
    [45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59,
     60, 61, 62, 63]
```

Interpreting this for `IV(16) || C1(16) || C2(16) || C3(16)`:

- Byte offsets 45, 46, 47 = last three bytes of **C2** (the block before the final ciphertext block). These directly control the final plaintext's trailing bytes, so flipping them breaks padding.
- Byte offsets 48..63 = **entire C3**. Flipping any of these randomizes the final block's decryption, destroying padding.

Conclusion: the plaintext ends with **three bytes of 0x03 padding** — PKCS#7 with pad length 3. The block structure is confirmed: 16-byte IV, three 16-byte ciphertext blocks.

Attack — CBC Padding Oracle

The math

For CBC decryption of a block `C_i`:

```
P_i = D_K(C_i) XOR C_{i-1}
```

Define the **intermediate value** `I_i = D_K(C_i)`. Then:

```
P_i = I_i XOR C_{i-1}
```

The oracle gives us one bit: "is the PKCS#7 padding of `P_i` valid?"

Strategy for the last byte of `I_i`:

1. Craft a modified previous block `C'` and query the oracle on `C' || C_i`.
2. Change only the last byte of `C'`, call it `g`. If the oracle returns valid, then `P_i[15] = g XOR I_i[15]` equals `0x01` (the only possible padding length when exactly one byte was manipulated). So `I_i[15] = g XOR 0x01`.
3. To recover byte 14, target padding `0x02 0x02`. Set `C'[15] = I_i[15] XOR 0x02` (forcing the last byte to `0x02`), then brute force `C'[14]` until the oracle accepts. `I_i[14] = g14 XOR 0x02`.

4. Continue for each position, targeting increasing padding lengths until all 16 bytes of `I_i` are recovered.
5. Finally `P_i = I_i XOR C_{i-1}` (or `I_i XOR IV` for the first block).

The attack script

Saved as `attack.py`:

```
#!/usr/bin/env python3
import subprocess, sys

BS = 16

def query(data):
    with open("/tmp/q.bin","wb") as f:
        f.write(data)
    return subprocess.run(["./oracle","/tmp/q.bin"],
                          capture_output=True).returncode == 0

def decrypt_block(prev, cur):
    inter = bytearray(BS)
    plain = bytearray(BS)
    for pad in range(1, BS+1):
        pos = BS - pad
        found = False
        for g in range(256):
            cc = bytearray(prev)
            cc[pos] = g
            for k in range(1, pad):
                cc[BS-k] = inter[BS-k] ^ pad
            if query(bytes(cc) + bytes(cur)):
                inter[pos] = g ^ pad
                plain[pos] = inter[pos] ^ prev[pos]
                found = True
                break
        if not found:
            print(f"  [!] no guess pos={pos} pad={pad}", file=sys.stderr)
            return None, None
    return bytes(plain), bytes(inter)

def attack(path):
    data = open(path,"rb").read()
    nb = len(data) // BS
    iv  = data[:BS]
    cbs = [data[i*BS:(i+1)*BS] for i in range(1, nb)]
    out = b""
    for i, cur in enumerate(cbs):
        prev = iv if i == 0 else cbs[i-1]
        print(f"[*] decrypting block {i+1}/{len(cbs)}", flush=True)
        p, _ = decrypt_block(prev, cur)
        if p is None:
            return None
        print(f"    -> {p!r}")
        out += p
```

```
    print(f"[+] plaintext: {out!r}")
    return out

if __name__ == "__main__":
    for path in (sys.argv[1:] or
                 ["ciphertext.bin","ciphertext2.bin"]):
        attack(path)
```

Running the attack

```
$ python3 attack.py ciphertext.bin ciphertext2.bin

[*] ciphertext.bin: 4 blocks (IV + 3 ciphertext blocks)
[*] decrypting block 1/3
    -> b'r00t{0r4cl3_p4dd'
[*] decrypting block 2/3
    -> b'1ng_6512e251d6e2'
[*] decrypting block 3/3
    -> b'8884bd5b37a0}\x03\x03\x03'
[+] plaintext: b'r00t{0r4cl3_p4dd1ng_6512e251d6e28884bd5b37a0}\x03\x03\x03'

[*] ciphertext2.bin: 5 blocks (IV + 4 ciphertext blocks)
[*] decrypting block 1/4
    -> b'\xcb\x95\\\x9b\xaa\x9a)_\xbd7&\xa5\x98\xcf\x12\x12'
...
[+] plaintext: b'\xcb\x95\\\x9b...'   (random - decoy)
```

The `\x03\x03\x03` at the end confirms PKCS#7 padding length 3 — matching the byte-flip detection earlier. `ciphertext.bin` is the flag. `ciphertext2.bin` is noise — a decoy.

Submitting

```
$ echo 'r00t{0r4cl3_p4dd1ng_6512e251d6e28884bd5b37a0}' | ./check
CORRECT
Flag: r00t{0r4cl3_p4dd1ng_6512e251d6e28884bd5b37a0}
```

`validator` rejected the raw plaintext with `INVALID - formula mismatch` — it expects the *intermediate value* formula, not the recovered plaintext. `check` is the scoring endpoint and accepts the plaintext directly.

What Worked

1. `strings` **on all three binaries first.** Immediately found the decoy flag in `oracle`, which told us the path forward was to *run* the oracle, not read the binary.
2. **Recognised exit-code-as-signal.** No stdout, but exit codes differ for valid vs invalid padding. This is the classic silent padding oracle interface.
3. **Byte-flip probe to locate the padding byte.** Flipping each byte and watching for the exit-code change pinned the padding length (3) and the block structure in a single pass.
4. **Standard CBC padding oracle attack.** 256 guesses × 16 bytes × N blocks. Ran in ~1 minute per ciphertext.

What Failed (and Why)

1. **Sending the full ciphertext per query.** The initial attack script treated each block as its own "ciphertext file" but fed the whole 64 bytes into the oracle. The oracle only inspects the *last* block's padding, so this failed for all but the last block. Fix: send `modified_prev || cur` (32 bytes) per query, one target block at a time.
2. **Assuming byte 15 was the padding byte.** First probe flipped only byte 15 (last byte of IV position). No change in exit code — because with three bytes of padding, modifying a byte far from the padding does nothing, and the actual padding bytes are at 45–47. Fix: flip *every* byte and look at the full set of positions that matter.
3. **Expecting `validator` to accept the plaintext.** `validator` wants a formula; its string `Formula verified: %s` indicates it checks some intermediate derivation (likely `I_i` values or a specific XOR chain). `check` is the final scoring endpoint and accepts the plaintext.

Real-World Impact — Where Padding Oracles Still Bite

Padding oracles have been a top-tier practical crypto bug for over two decades:

- **POODLE (CVE-2014-3566)** — padding oracle in SSL 3.0's CBC mode; forced browser downgrade to SSL 3.0 to exploit.
- **Lucky13 (CVE-2013-0169)** — timing-based padding oracle in TLS CBC; affected OpenSSL, GnuTLS, NSS. Still relevant for legacy TLS CBC suites.
- **BEAST (CVE-2011-3389)** — related CBC chosen-plaintext attack against TLS 1.0.
- **ASP.NET padding oracle (2010)** — Microsoft's `ScriptResource.axd` leaked padding oracle, allowing attackers to decrypt viewstate and auth cookies.
- **Java JSF / Oracle ADF** — repeated padding oracle disclosures through the 2010s in enterprise frameworks.
- **Node.js `cookie` and various JWT libraries** — cases where CBC-mode tokens or session cookies leaked padding behaviour via error messages or response timing.

Where to look in bug bounties

1. **Any encrypted cookie, token, or session value** that comes back with a decrypt/verify error path differing from other errors. Test with bit-flipped ciphertext — if the response changes (status code, body, timing, error message), you likely have an oracle.
2. **Encrypted URL parameters** (`?id=<base64>`). Flip the last byte of the decoded ciphertext and watch for distinct error responses.
3. **"Remember me" tokens** with CBC-mode encryption (legacy Rails, Django, old Spring).
4. **Error-based oracles** — "invalid padding" vs "invalid signature" vs "malformed token". Any distinct error is a bit.
5. **Timing side-channels** — Lucky13-class bugs. Measure response time over thousands of requests with a `hrtime`-grade clock.

The core lesson

Do not reveal *why* a decryption failed. Return the same generic error, at the same time, regardless of whether padding, MAC, or ciphertext was the problem. Better: authenticate-then-decrypt (encrypt-then-MAC), and use AEAD modes (AES-GCM, ChaCha20-Poly1305) that don't have a padding oracle surface at all.

Fixes (Defensive)

1. **Use AEAD.** AES-GCM or ChaCha20-Poly1305 eliminates the padding oracle class entirely. No padding, integrity checked during decryption.
2. **Encrypt-then-MAC.** If CBC is unavoidable, MAC the ciphertext and verify the MAC *before* attempting decryption. An attacker who flips a byte in the ciphertext fails MAC verification and never reaches the padding step.
3. **Constant-time error handling.** Every decryption failure must return the same response, at the same time. No "invalid padding" distinct from "invalid MAC".
4. **Constant-time decryption routines.** Prevent Lucky13-class timing oracles.
5. **Rotate keys and tokens on suspicion.** If a padding oracle is discovered in production, assume all encrypted data that passed through it is compromised.

Key Takeaways

- **A single bit of leaked information is enough to break CBC encryption.** The padding oracle isn't a math weakness in AES — it's a protocol design bug that lets an attacker recover plaintext byte-by-byte.
- **Exit codes, error strings, and timing all leak.** "Silent" binaries that only differ in exit status are still oracles.
- **Attack in the right granularity.** CBC padding oracles operate on the *last block* of whatever you feed in — you must craft `prev || target` per query, not send the full ciphertext.
- **Decoys are everywhere.** The `oracle` binary contained a fake flag designed to short-circuit solvers who `strings`-grepped and stopped. `ciphertext2.bin` decrypted to noise on purpose. Confirm everything.
- **The formula the challenge wants isn't always the plaintext.** The `validator` binary wanted an intermediate derivation; `check` wanted the plaintext. Read the strings of every binary before deciding what to submit.

Tools Used

- `strings`, `file`, `xxd` — static recon
- `python3` + `subprocess` — query loop and byte-flip probe
- Custom CBC padding oracle attack script (`attack.py`)

Commands That Worked

```
# static recon
strings oracle validator check | grep -iE "flag|usage|padding|valid|formula"

# confirm exit-code oracle
./oracle ciphertext.bin ; echo $?
./oracle --mode 2 ciphertext.bin ; echo $?

# locate padding byte via byte-flip probe
python3 flip_probe.py    # reports offsets [45..63]

# run the attack
python3 attack.py ciphertext.bin ciphertext2.bin

# submit
echo 'r00t{...}' | ./check
```

Commands That Did NOT Work (and why)

```
# Sending the whole ciphertext per query - only last block's padding is checked
./oracle ciphertext.bin
# Fix: send modified_prev || target_block

# Flipping only byte 15 - misses padding bytes at 45-47 when pad length > 1
# Fix: flip every byte, look at the full hit set

# Feeding plaintext to validator
echo 'r00t{...}' | ./validator     # INVALID - formula mismatch
# Fix: validator wants a derivation; check accepts plaintext
```